



Язык программирования C++

Алгоритмы и предикаты.

Преподаватели:

Пысин Максим Дмитриевич, ассистент кафедры ИКТ

Лобанов Алексей Владимирович, аспирант кафедры ИКТ

Крашенинников Роман Сергеевич, аспирант кафедры ИКТ



Стандартная библиотека



В языке программирования C++ термин **Стандартная Библиотека** означает коллекцию классов и функций, реализованных на самом языке.

Стандартные C++ библиотеки представляют собой наборы функций, констант, классов, объектов и шаблонов, которые расширяют язык C++, предоставляя базовую функциональность для выполнения различных задач, таких как: классы для взаимодействия с операционной системой, контейнеры данных, манипуляторы для работы с этими данными и наиболее используемые алгоритмы.

Все элементы стандартных библиотек C++ распределены по различным заголовочным файлам, которые необходимо подключать к программе. Только так можно использовать элементы этих библиотек. Ниже приведены стандартные библиотеки языка программирования C++.

Стандартная Библиотека поддерживает несколько основных контейнеров, функций для работы с этими контейнерами, объектов-функции, основных типов строк и потоков (включая интерактивный и файловый ввод-вывод), поддержку некоторых языковых особенностей, и часто используемые функции для выполнения таких задач, как, например, нахождение квадратного корня числа.

Стандартная библиотека шаблонов (STL) — подмножество стандартной библиотеки C++ и содержит контейнеры, алгоритмы, итераторы, объекты-функции и т. д. Хотя некоторые программисты используют термин «STL» вместе (или попеременно) с термином «Стандартная библиотека C++».

Состав стандартной библиотеки



В стандартной библиотеке C++ выделяют следующие компоненты:

- **Language support library** – отвечает за языковую поддержку
- **Diagnostics library** – отвечает за исключения(ошибки времени исполнения)
- **General utilities library** – отвечает за утилиты общего характера
- **Strings library** – отвечает за работу со строками
- **Localization library** – отвечает за локализацию программы
- **Containers library** – отвечает за все варианты контейнеров и адаптеров
- **Iterators library** – отвечает за итераторы для работы с контейнерами
- **Algorithms library** – отвечает за стандартные алгоритмы
- **Numerics library** – отвечает за работу с числами
- **Input/output library** – отвечает за ввод/вывод информации
- **Regular expressions library** – отвечает за регулярные выражения
- **Atomic operations library** – отвечает за атомарные операции
- **Thread support library** – отвечает за многопоточность

В стандартной библиотеке шаблонов выделяют пять основных компонентов:

- **Контейнер** (container)
- **Итератор** (iterator)
- **Алгоритмы** (algorithm)
- **Адаптеры** (adaptor)
- **Функциональные объекты** (functor)



Библиотека алгоритмов



Алгоритм — конечная совокупность точно заданных правил решения произвольного класса задач или набор инструкций, описывающих порядок действий исполнителя для решения некоторой задачи.

Методы сортировки коллекции:

- `sort` - сортировка,
- `stable_sort` – стабильная сортировка,
- `partial_sort` – частичная сортировка,
- `partial_sort_copy` – частичная сортировка с копированием,
- `nth_element` – группировка элементов больше и меньше определенного

Работа с отсортированными коллекциями:

- `binary_search` – бинарный поиск
- `lower_bound` – поиск первого не меньшего элемента упорядоченной последовательности
- `upper_bound` – поиск первого большего элемента упорядоченной последовательности
- `merge` – объединение отсортированных последовательностей и запись в место указанное для результата
- `inplace_merge` – объединение последовательностей и запись в начало объединяемых последовательностей
- `includes` – проверяет включает ли одна последовательность другую
- `min` – минимальное значение
- `max` – максимальное значение
- `min_element` – минимальный элемент
- `max_element` – максимальный элемент



Работа с коллекциями



Методы перебора всех элементов коллекции и их обработки:

- `count` – подсчет элементов по значению
- `count_if` – подсчет по условию
- `find` – поиск по значению
- `find_if` – поиск по условию
- `adjacent_find` – поиск одинаковых соседних пар
- `for_each` – перебор всех элементов и применение к ним функции
- `mismatch` – поиск первого различающегося элемента
- `equal` – сравнение на равенство последовательностей
- `copy` - копирование
- `copy_backward` – обратное копирование
- `swap` – обмен элементов по значению
- `iter_swap` – обмен элементов через указатели
- `swap_ranges` – обмена диапазонов
- `fill` – заполнение последовательности значениями
- `fill_n` – заполнение N элементов контейнера значениями
- `generate` – вызов функции генератора для заполнения каждого элемента последовательности
- `generate_n` – заполнение N элементов при помощи генератора



Работа с коллекциями



Методы перебора и манипуляции элементами коллекции:

- `replace` – замена одного значения на другое в интервале
- `replace_if` – замена по условию
- `transform` – преобразование каждого элемента последовательности
- `remove` – удаление элемента по значению в интервале
- `remove_if` – удаление элементов по условию,
- `remove_copy` – копирование элементов с игнорированием по значению,
- `remove_copy_if` – копирование с игнорированием по условию,
- `unique` – получение уникальной последовательности
- `unique_copy` – копирование только уникальных значений
- `reverse` – инверсия последовательности
- `reverse_copy` – копирование обратной последовательности
- `rotate` – поворот последовательности по направлению
- `rotate_copy` – поворот последовательности с копированием
- `random_shuffle` – перемешивание последовательности
- `partition` – разбиение на диапазоны по условию



А где сигнатуры?



Приводить сигнатуры для каждого из перечисленных методов слишком расточительно по месту на слайдах, поэтому, просто пользуйтесь сайтом cplusplus: <https://en.cppreference.com/>

std::sort

Defined in header <algorithm>

```
template< class RandomIt >                                (until C++20)
void sort( RandomIt first, RandomIt last );                (1)
template< class RandomIt >                                (since C++20)
constexpr void sort( RandomIt first, RandomIt last );

template< class ExecutionPolicy, class RandomIt >          (2) (since C++17)
void sort( ExecutionPolicy&& policy,
          RandomIt first, RandomIt last );

template< class RandomIt, class Compare >                  (until C++20)
void sort( RandomIt first, RandomIt last, Compare comp ); (3)
template< class RandomIt, class Compare >                  (since C++20)
constexpr void sort( RandomIt first, RandomIt last, Compare comp );

template< class ExecutionPolicy, class RandomIt, class Compare > (4) (since C++17)
void sort( ExecutionPolicy&& policy,
          RandomIt first, RandomIt last, Compare comp );
```

Sorts the elements in the range [first, last) in non-decreasing order. The order of equal elements is preserved.

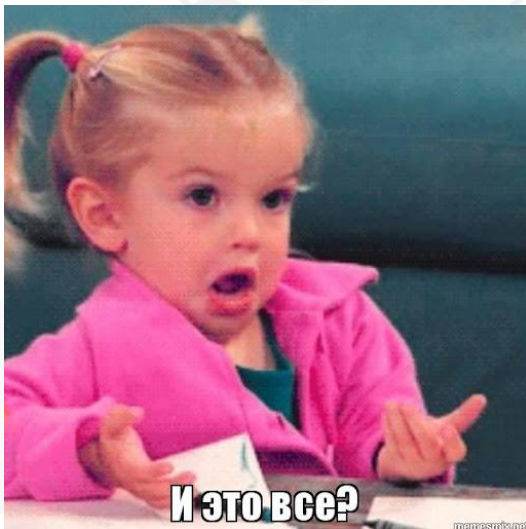


Разбирать все
функции

Отправить
читать доку

На этом сайте есть все элементы стандартной библиотеки, от контейнеров и алгоритмов, до библиотеки параллельных потоков. Практически на каждой странице есть пример, описание искомого метода или класса, а так же его ограничения и особенности.

Пример



```
int main(){  
    vector<int> v(10);  
    for(auto& el: v){  
        el = -50 + rand() % 100;  
        cout << el << ", ";  
    }  
    cout << endl;  
    sort(v.begin(), v.end());  
    for(auto el: v){  
        cout << el << ", ";  
    }  
    cout << endl;  
}
```

-9, 17, -16, -50, 19, -26, 28, 8, 12, 14,
-50, -26, -16, -9, 8, 12, 14, 17, 19, 28,

Работа с пользовательским типом



```
class MyInt{
private:
    int value = 0;
public:
    int getValue(){
        return value;
    }
    void operator=(int v){
        value = v;
    }
    friend ostream& operator<<(ostream& os, MyInt& v);
};
ostream& operator<<(ostream& os, MyInt& v){
    return os << v.value;
}
```



```
int main(){
    vector<MyInt> v(10);
    for(auto& el: v){
        el = -50 + rand() % 100;
        cout << el << ", ";
    }
    cout << endl;
    sort(v.begin(), v.end());
    for(auto el: v){
        cout << el << ", ";
    }
    cout << endl;
}
```

D:/5-Programs/MinGW64_8.1/mingw64/lib/gcc/x86_64-w64-mingw32/8.1.0/include/c++/bits/predefined_ops.h:65:22: note: 'MyInt' is not derived from 'const __gnu_cxx::__normal_iterator<Iterator, _Container>'

```
{ return *__it < __val; }
```

Предикат



Предикат — функция, возвращающая bool, также это может быть функтор оператор () которого возвращает bool. Унарный предикат — предикат принимающий 1 аргумент, к примеру !a. Бинарный предикат — предикат, принимающий 2 аргумента, примеры: $a > b$, $a < b$ и др.

```
bool compareMyInt(MyInt& a, MyInt& b){
    return a.getValue() > b.getValue();
}

int main(){
    vector<MyInt> v(10);
    for(auto& el: v){
        el = -50 + rand() % 100;
        cout << el << ", ";
    }
    cout << endl;
    sort(v.begin(), v.end(), compareMyInt);
    for(auto el: v){
        cout << el << ", ";
    }
    cout << endl;
}
```

-9, 17, -16, -50, 19, -26, 28, 8, 12, 14,
28, 19, 17, 14, 12, 8, -9, -16, -26, -50,



Функтор

Функторы в C++ являются сокращением от "функциональные объекты". Функциональный объект является экземпляром класса C++, в котором определён operator(). Если вы определите operator() для C++ класса, то вы получите объект, который действует как функция, но может также хранить состояние.

```
class CompareMyIntFunctor{
    int calcs = 0;
public:
    bool operator()(MyInt& a, MyInt& b){
        cout << calcs << endl;
        calcs++;
        return a.getValue() > b.getValue();
    }
    int getCals(){
        return calcs;
    }
};
```

```
int main(){
    vector<MyInt> v(10);
    for(auto& el: v){
        el = -50 + rand() % 100;
        cout << el << ", ";
    }
    cout << endl;
    CompareMyIntFunctor functor;
    cout << functor.getCals() << endl;
    sort(v.begin(), v.end(), functor);
    for(auto el: v){
        cout << el << ", ";
    }
    cout << endl;
    cout << functor.getCals() << endl;
}
```



Лямбда



```
int main(){
    vector<MyInt> v(10);
    for(auto& el: v){
        el = -50 + rand() % 100;
        cout << el << ", ";
    }
    cout << endl;
    int calcs = 0;
    cout << calcs << endl;
    sort(v.begin(), v.end(), [&calcs](MyInt& a, MyInt&b){
        calcs++;
        return a.getValue() > b.getValue();
    });
    for(auto el: v){
        cout << el << ", ";
    }
    cout << endl;
    cout << calcs << endl;
}
```

-9, 17, -16, -50, 19, -26, 28, 8, 12, 14,
0
28, 19, 17, 14, 12, 8, -9, -16, -26, -50,
31



Что это такое?



Понятие лямбда-функции пришло из математической логики и функционального программирования, а корни уходят в работы **Алонзо Чёрча 1930-х годов**. Он разработал формальную систему под названием **λ-исчисление** (lambda calculus) — это математическая модель, описывающая, как можно выражать вычисления с помощью функций и их применения к аргументам.

Ключевой идеей модели является посылка, что любое вычисление можно представить как последовательность применения функций к значениям.

Функциональные языки, такие как Lisp (1958), ML (1973), Scheme (1975) и Haskell (1990), строились вокруг принципа, что функции — это значения. В таком случае их можно:

- **создавать прямо во время выполнения программы,**
- **передавать как аргументы другим функциям,**
- **возвращать из функций.**

С другой стороны функции, которые принимают функции, возвращают функции или делают и то и другое называют **функциями высшего порядка (higher-order function)**. Это одно из ключевых понятий в функциональном программировании и напрямую связано с лямбда-функциями

LISP: `(map (lambda (x) (+ x 1)) '(1 2 3))` - применяют функцию “+1” ко всем элементам списка.



А в C++?



До стандарта C++11 в языке не было встроенных анонимных функций — их заменяли функторы и указатели на функции. Однако, под влиянием функциональных концепций, появившихся в других языках, **стандарт C++11 ввёл лямбда-выражения**, которые реализуют ту же идею — **создание безымянной функции прямо в месте использования**.

Их развитие продолжилось в C++14, C++17 и C++20, где были добавлены новые возможности.

```
std::for_each(v.begin(), v.end(), [](int& x) { x *= 2; });
```

Лямбда-функции — это мост между математическими принципами функционального программирования и практической реализацией в C++. Они позволяют использовать выразительные, компактные и безопасные функциональные конструкции внутри императивного и объектно-ориентированного языка.

Лямбда-функции не могли бы иметь такого значения без **замыканий**.



Замыкание?



Замыкание (closure) — это функция, которая “захватывает” переменные из своей внешней области **видимости** и может использовать их даже после того, как эта область закончила своё существование. Каждое лямбда-выражение в C++ создаёт **объект особого безымянного типа**, который захватывает переменные, поэтому процесс расширения контекста называю **захватом(capture)**.

Технически:

- Каждая лямбда создаёт объект специального типа с перегруженным оператором `operator()`.
- Захваченные переменные становятся членами этого объекта.
 - При захвате по значению — копии значений сохраняются внутри.
 - При захвате по ссылке — хранятся ссылки на внешние объекты.
 - Если переменная, захваченная по ссылке, **уничтожена** до вызова лямбды, поведение **неопределено**.
- Объект имеет тип `non-union, non-aggregate`
- Лямбду без захваченных переменных можно **неявно преобразовать в указатель на функцию**

На практике:

- создавать компактные обработчики и фильтры без глобальных переменных;
- хранить контекст состояния между вызовами;
- передавать в функции поведение, зависящее от внешних переменных;
- строить функции высшего порядка, которые возвращают другие функции с уже частично “зашитым” контекстом

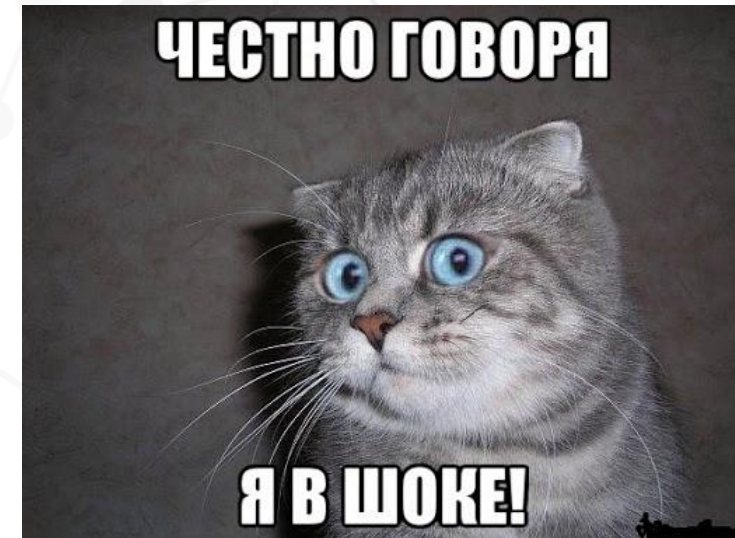
Сигнатура лямбды



Лямбда выражения – техника программирования, сочетающая в себе преимущества указателей на функции и функциональных объектов, позволяет избежать неудобств. Лямбда выражения позволяют вам писать более компактный код и избежать ошибок, нежели используя функциональные объекты.

```
[<захваченные переменные>] (<аргументы функции>)  
  ③ mutable  
  ④ noexcept  
  ⑤ -> <тип возвращаемого значения>  
{  
    // Тело функции ⑥  
}
```

- 1) Захват переменных
- 2) Список аргументов функции
- 3) Спецификатор изменения параметра переданного по значению
- 4) Спецификатор отсутствия исключений
- 5) Тип возвращаемого значения
- 6) Тело лямбда выражения



Части сигнатуры



Захват переменных

Лямбда выражение может получать доступ практически к любым переменным и использовать их внутри тела. Маска определяет способ получения параметров телом лямбда выражения. Доступ к переменным, перед которыми стоит амперсанд (&), осуществляется по ссылке, а перед которыми нет амперсанда, соответственно по значению. Пустая маска ([]) означает, что тело выражения не имеет доступа к переменным.

- [] ничего не захватывает;
- [a,&b] где a захвачена по значению, а b захвачена по ссылке;
- [this] захватывает указатель this по значению;
- [&] захват всех символов по ссылке;
- [=] захват всех символов по значению;
- [=, &b] по умолчанию по значению, y по ссылке;
- [&, a] по умолчанию по ссылке, x по значению;

С C++ 14 можно инициализировать переменные внутри маски

```
[v = std::vector<int>{1,2,3}]() { return v.size(); };
```

А с C++ 17, можно захватывать объект класса по значению: [*this].



Части сгнатуры



Список аргументов функции

Список параметров лямбда выражения содержит список параметров для функции, у которых существуют следующие ограничения:

1. Список параметров не может содержать значения по умолчанию
2. Не может содержать неименованные параметры
3. Ограниченное число параметров

```
[ ](int a, int b) { return a + b; };  
[ ](const std::string& s) { std::cout << s; };
```

В стандарте **C++14** появилась возможность определять аргументы лямбда функций как `auto`, за счёт чего лямбда функцию можно передавать в качестве аргумента самой себе по ссылке.

```
[ ](auto a, auto b) { return a + b; };
```

В C++20 появилась возможность использовать концепты.

```
[ ](std::integral auto x) { return x + 1; };
```



Части сигнатуры



Спецификатор изменение параметра переданного по значению

Спецификация позволяет разрешить телу лямбда выражения и заменить переменные, полученные по значению.

Спецификатор отсутствия исключений

Лямбда выражения может включать в себя спецификацию отсутствия исключения. Ключевое слово `noexcept` указывает компилятору, может ли функция (в том числе лямбда) выбрасывать исключения. Если функция не должна выбрасывать исключений, её объявляют как `noexcept`. Если во время выполнения из такой функции всё же выбрасывается исключение, программа вызывает `std::terminate()` — аварийное завершение.

Зачем?

- Безопасность и предсказуемость
- Оптимизация производительности
- Контракт интерфейса

```
[]() noexcept {  
    throw std::runtime_error("oops"); // X  
    приведёт к std::terminate()  
};
```

Возможно указать условие действительность `noexcept`: `noexcept(sizeof(int) == 4)`

Имеет влияние на использование `move` семантики в контейнерах стандартной библиотеки.

Части сигнатуры



Возвращаемый тип

Спецификация определяет возвращаемый тип лямбда выражения. Вы можете не использовать данную конструкцию, если тело выражения имеет только одну инструкцию “**return**” или не возвращает значения вообще. Если тело содержит только одну “**return**” инструкцию, компилятор установит возвращаемый тип лямбда выражения идентичный типу “**return**” инструкции.

Это означает, что лямбда функция может использовать не только переменные, которые передаются ей в качестве параметров, но и какие-либо объекты, которые были объявлены вне **лямбда-функции**.

Тело лямбда выражения

На тело выражения накладываются идентичные ограничения, как на блок кода или метод.

requires

Со стандарта C++ 20 появилась возможность создавать шаблонизированные лямбды и использовать внутри них требования к типам спецификации шаблонов через концепты или блок `requires`.

```
[<typename Container, typename Func, typename Pred>
    (Container& c, Func f, Pred p) requires std::invocable<Func,
typename Container::value_type>
{...}
```

Пример



```
int main(){
    srand(time(0));
    vector<int> v(100);
    for(auto& el: v){
        el = -50 + rand() % 100;
    }
    int positives = count_if(v.begin(), v.end(), [](int& el){ return el > 0; });
    int negatives = v.size() - positives;
    cout << positives << " vs " << negatives << endl;
    for_each(v.begin(), v.end(),
        [positives, negatives] (auto& el) mutable{
            // cout << el << " " << positives << " " << negatives << endl;
            if(el > 0 && positives > negatives){
                el *= -1;
                positives--;
                negatives++;
            }
            if(el < 0 && positives < negatives){
                el *= -1;
                positives++;
                negatives--;
            }
        }
    );
    cout << positives << " vs " << negatives << endl;
    positives = count_if(v.begin(), v.end(), [](int& el){ return el > 0; });
    negatives = v.size() - positives;
    cout << positives << " vs " << negatives << endl;
}
```

62 vs 38
62 vs 38
50 vs 50

Генерация случайных чисел



Мы привыкли использовать генератор случайных чисел из C, так как он прост и понятен, нам он давно известен. Однако, он имеет ряд ограничений:

- Он генерирует только целочисленные значения, а для получения дробных значений выполняется деление на максимально возможное сгенерированное значение.
- Он использует только один алгоритм генерации, предоставляющий равномерное распределение чисел по всему диапазону.

Для ряда задач эти ограничения могут быть критичными, по этой причине, в стандартной библиотеке C++ **<random>** определены собственные способы генерации случайных чисел.

Генерация в C++ делится на 3 части:

1. Выбор основы для генератора случайных чисел – это зерно генератора, на основании которого будет строиться последовательность. Зерном является random device детерминированный генератор целочисленных значений, точнее результат его работы
2. Выбор алгоритма генерации, библиотека предоставляет множество вариантов генерации, один из самых популярных mt19937 (чаще всего встречается в примерах на C++).
3. Выбор распределения, стандартное равномерное распределение uniform int distribution (для целых чисел) и uniform real distribution (для рациональных).

Пример



```
void print(vector<int>& v){
    const int VN = v.size();
    int i = 0;
    cout << "vector[" << VN << "]{";
    for_each(
        v.begin(),
        v.end(),
        [&i, &VN](auto& el){
            cout << el << ((i++ != VN - 1) ? ", ": "{")");
        }
    );
    cout << endl;
}

void print(list<double>& l){
    const int LN = l.size();
    int i = 0;
    cout << "list[" << LN << "]{";
    for_each(
        l.begin(),
        l.end(),
        [&i, &LN](auto& el){
            cout << setprecision(3) << el
                << ((i++ != LN - 1) ? ", ": "{")");
        }
    );
    cout << endl;
```

```
const int N = 20;
```

```
int main(){
    vector<int> v;
    list<double> l;
    random_device r;
    mt19937 engine(r());
    uniform_int_distribution<int> ind(-100, 100);
    uniform_real_distribution<double> dnd(-1., 1.);
    for(int i = 0; i < N; i++){
        v.push_back(ind(engine));
        l.push_back(dnd(engine));
    }
    print(v);
    print(l);
    return 0;
}
```

```
vector[20]{12, 83, 75, 96, -37, 58, 15, 17, -66, -97, -4, -32, 69,
30, 70, -43, -100, -9, 87, 44}
list[20]{-0.303, 0.189, 0.918, 0.323, 0.403, -0.852, -0.324, -
0.938, 0.932, 0.559, -0.868, -0.17, -0.434, 0.141, 0.121, 0.459, -
0.476, -0.553, -0.0438, 0.771}
```

Унифицируем пример



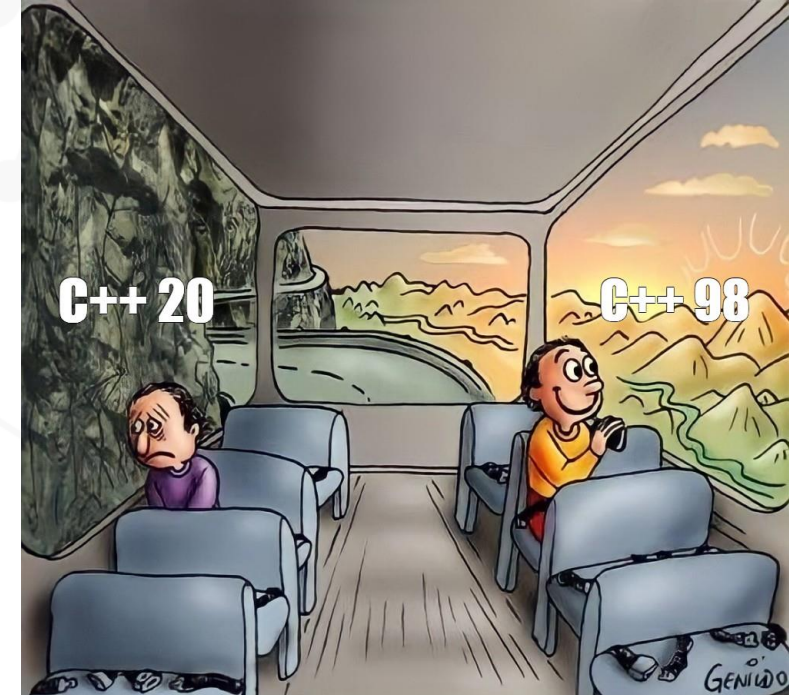
```
template <typename Type, class Container>
Container generate(
    const int& N,
    const Type& left = std::is_floating_point<Type>::value ? -1.: -100,
    const Type& right = std::is_floating_point<Type>::value ? 1.: 100
){
    std::random_device r;
    std::mt19937 engine(r());
    Container container;
    if constexpr (std::is_floating_point<Type>::value){
        std::uniform_real_distribution<Type> dist(left, right);
        for(int i = 0; i < N; i++) container.push_back(dist(engine));
    } else {
        std::uniform_int_distribution<Type> dist(left, right);
        for(int i = 0; i < N; i++) container.push_back(dist(engine));
    }
    return container;
}
```



Унифицируем пример

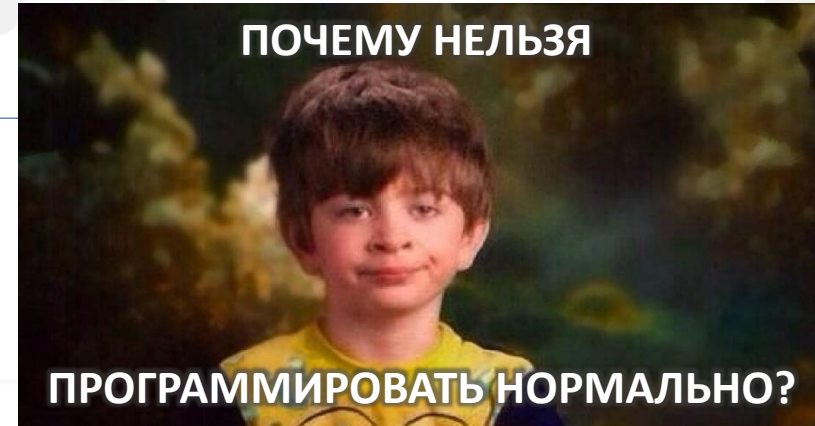


```
template <
    typename Type,
    template <typename...> class Container
>
requires std::is_same_v<typename Container<Type>::value_type, Type> &&
    (std::integral<Type> || std::floating_point<Type>)
auto generate(
    int N,
    Type left  = std::is_floating_point_v<Type> ? static_cast<Type>(-1) : static_cast<Type>(-100),
    Type right = std::is_floating_point_v<Type> ? static_cast<Type>(1)  : static_cast<Type>(100)
) -> Container<Type>
{
    std::random_device r;
    std::mt19937 engine(r());
    Container<Type> container;
    if constexpr (std::floating_point<Type>) {
        std::uniform_real_distribution<Type> dist(left, right);
        for (int i = 0; i < N; ++i)
            container.push_back(dist(engine));
    } else {
        std::uniform_int_distribution<Type> dist(left, right);
        for (int i = 0; i < N; ++i)
            container.push_back(dist(engine));
    }
    return container;
}
```

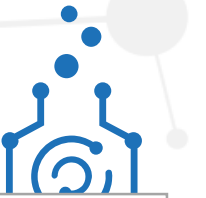


Унифицируем пример

```
template <class Container>
void print(const Container& v){
    const int VN = v.size();
    int i = 0;
    std::string name = __PRETTY_FUNCTION__; // не средство языка, а компилятора
    name = name.substr(name.find_last_of("::") + 1, name.find("]") - 1);
    std::cout << name << "[" << VN << "]{";
    std::for_each(
        v.begin(),
        v.end(),
        [&i, &VN](auto& el){
            std::cout << el << ((i++ != VN - 1) ? ", ": "});
        }
    );
    std::cout << std::endl;
}
```



Унифицируем пример



```
#include <vector>
#include <list>
#include <set>
#include <deque>

#include "base.h"

using namespace std;

const int N = 20;

int main(){
    vector<int> v = generate<int, vector<int>>>(N);
    list<double> l = generate<double, list<double>>>(N);
    deque<long long int> d = generate<long long int, deque<long long int>>>(N, -
1000000000000, 1000000000000);

    print(v);
    print(l);
    print(d);
    return 0;
}
```

```
vector<int>[20]{12, 45, -30, 83, -99, 19, 75, 3, 92, 96, -66, 32, -37, 6, 40, 58, -49, -86, 15,
-34}
list<double>[20]{0.452498, 0.830679, 0.188648, 0.0307165, 0.950299, 0.323121,
0.0573049, 0.576986, -0.851799, -0.3403, 0.167488, -0.938056, 0.857187, -0.960495,
0.558744, -0.32791, -0.314948, -0.170361, 0.814626, 0.302961}
deque<long long int>[20]{124105681002, -300283945447, -988857462583,
745970169002, 919743648206, -657856202574, -366369262418, 403545890237, -
492875599044, 148172963230, -323182973939, -523125605698, -652414337696,
932820119257, 309017435041, -45073958523, -869679843303, -972448559986,
687524060525, -434561197696}
```

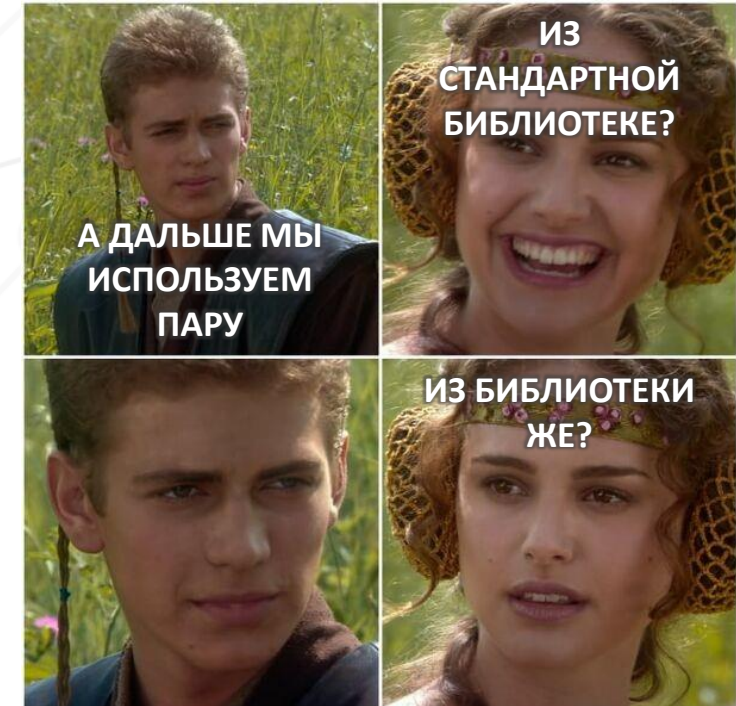
Пример с сортировками



```
template <typename FT, typename ST>
class Pair:public pair<FT, ST>{
public:
    Pair(const FT& a, const ST& b): pair<FT, ST>(a, b) {};
    template <typename FFT, typename FST>
    friend ostream& operator<<(ostream& out, const Pair<FFT, FST>& value);
};
```

```
template <typename FFT, typename FST>
ostream& operator<<(ostream& out, const Pair<FFT, FST>& value){
    return out << "(" << value.first << "," << value.second << ")";
}
```

```
bool equal_order(vector<Pair<int, int>> a, vector<Pair<int, int>> b){
    return std::equal(
        a.begin(), a.end(), b.begin(),
        [](const Pair<int, int>&a, const Pair<int, int>&b){
            return a.first == b.first && a.second == b.second;
        }
    );
}
```



Пример с сортировками



```
int main(){
    vector<int> v = generate<int, vector<int>>(N, 0, 10);
    vector<Pair<int, int>> v_pair;
    int i = 0;
    for(int el: v) v_pair.push_back(Pair<int, int>(el, i++));
    vector<int> v_copy = v;
    vector<Pair<int, int>> v_pair_copy = v_pair;
    print(v_copy);
    sort(v_copy.begin(), v_copy.end());
    print(v_copy);
    print(v_pair_copy);
    sort(v_pair_copy.begin(), v_pair_copy.end(), [](const Pair<int, int>& a, const Pair<int, int>& b){return a.first < b.first;});
    print(v_pair_copy);
    ...
    cout << endl;
    vector<Pair<int, int>> v_pair_t_copy = v_pair;
    print(v_pair_t_copy);
    stable_sort(v_pair_t_copy.begin(), v_pair_t_copy.end(), [](const Pair<int, int>& a, const Pair<int, int>& b){return a.first <
b.first;});
    print(v_pair_t_copy);

    cout << "equal order first(sort), third(stable_sort): " << equal_order(v_pair_copy, v_pair_t_copy) << endl;
    ...
    return 0;
}
```

Пример с сортировками



```
vector<int>][20]{6, 6, 0, 8, 7, 9, 10, 5, 1, 6, 9, 4, 8, 7, 1, 2, 5, 0, 1, 0}
vector<int>][20]{0, 0, 0, 1, 1, 1, 2, 4, 5, 5, 6, 6, 6, 7, 7, 8, 8, 9, 9, 10}
vector<Pair<int, int> >][20]{(6,0), (6,1), (0,2), (8,3), (7,4), (9,5), (10,6), (5,7), (1,8), (6,9), (9,10), (4,11), (8,12), (7,13), (1,14), (2,15), (5,16), (0,17), (1,18), (0,19)}
vector<Pair<int, int> >][20]{(0,19), (0,2), (0,17), (1,18), (1,8), (1,14), (2,15), (4,11), (5,16), (5,7), (6,1), (6,9), (6,0), (7,13), (7,4), (8,12), (8,3), (9,10), (9,5), (10,6)}
equal order first(sort), second(sort): 1

vector<Pair<int, int> >][20]{(6,0), (6,1), (0,2), (8,3), (7,4), (9,5), (10,6), (5,7), (1,8), (6,9), (9,10), (4,11), (8,12), (7,13), (1,14), (2,15), (5,16), (0,17), (1,18), (0,19)}
vector<Pair<int, int> >][20]{(0,2), (0,17), (0,19), (1,8), (1,14), (1,18), (2,15), (4,11), (5,7), (5,16), (6,0), (6,1), (6,9), (7,4), (7,13), (8,3), (8,12), (9,5), (9,10), (10,6)}
equal order first(sort), third(stable_sort): 0

vector<Pair<int, int> >][20]{(6,0), (6,1), (0,2), (8,3), (7,4), (9,5), (10,6), (5,7), (1,8), (6,9), (9,10), (4,11), (8,12), (7,13), (1,14), (2,15), (5,16), (0,17), (1,18), (0,19)}
vector<Pair<int, int> >][20]{(0,2), (0,17), (0,19), (1,8), (1,14), (1,18), (2,15), (4,11), (5,7), (5,16), (6,0), (6,1), (6,9), (7,4), (7,13), (8,3), (8,12), (9,5), (9,10), (10,6)}
equal order third(stable_sort), fourth(stable_sort): 1
```

КРЕПИТЕСЬ

КОНЕЦ БЛИЗОК

HBO Game of Thrones™

risovach.ru

Спасибо за внимание